

The Poisson Mode of GBDTE: What It Is, How It Works, and What Is Broken

Package documentation — `golang/poisson_legacy` and
`python/extra_boost_py/experiments/poisson_bench.py`

July 3, 2026

Abstract

This document explains, from first principles, what the Poisson mode of the GBDTE package models, how the legacy Go implementation trains it, how the new synthetic benchmark generator produces data for it, and — based on the empirical validation of 2026-07-02 — where the implementation is inconsistent and why. It is meant as the shared mental model before fixing the engine. Statements about the code cite exact files and functions; statements about behavior cite the reproducible tests in `python/tests/test_poisson_semantics.py`.

1 The problem the Poisson mode solves

1.1 Counts instead of values or labels

The package’s other two losses answer “what value?” (MSE) and “which class?” (LogLoss). The Poisson mode answers “*how many events, and at what rate?*”: insurance claims per policy-year, machine failures per month, calls per hour, air-traffic conflicts per sector-day. The classical tool is Poisson regression, a generalized linear model in which a count y is modeled as

$$y \sim \text{Poisson}(\mu), \quad \mu = \mathbb{E}[y \mid x],$$

and the model is fitted by maximizing the Poisson log-likelihood $\sum_i (y_i \log \mu_i - \mu_i)$ (up to a term not depending on the model) [1, 2]. Modern gradient-boosting packages all offer this as an objective: LightGBM (`objective=poisson`), XGBoost (`count:poisson`), CatBoost (`loss_function=Poisson`) [9, 10, 11]; in actuarial practice Poisson-boosted trees are the standard workhorse for claim frequency [5, 6, 7, 8].

1.2 The GBDTE twist: intensity that drifts in time

All of the above predict a *constant* expected count per feature vector. GBDTE’s premise is different: the event rate is a *function of time*,

$$\lambda(t \mid x) = \text{events per unit time for an object with features } x,$$

and what we care about is *extrapolating* λ into a future time window that the training data never saw. This is the inhomogeneous Poisson process setting [3]: over a window W , an object with intensity $\lambda(t)$ produces $\text{Poisson}(\int_W \lambda(t) dt)$ events, and the log-likelihood of observed event times $t_1, \dots, t_n$ is

$$\ell = \sum_{i=1}^n \log \lambda(t_i) - \int_W \lambda(t) dt. \tag{1}$$

GBDTE applies its usual role separation to this problem:

- **partition features** $X^{(p)}$ (stable object attributes) drive the tree splits and identify *which* intensity regime an object belongs to;
- **extrapolation basis** $\varphi(t)$ (e.g. $(1, t)$) lives inside each leaf: the leaf stores a weight vector w and predicts the time-varying intensity

$$\lambda(t) = w^\top \varphi(t)$$

— a *linear intensity model per leaf*, so the fitted rate has a trend that can be carried into the future window.

A conventional Poisson booster puts t among the ordinary features and therefore predicts a piecewise-constant function of t ; beyond the last training split it extrapolates a constant. GB-DTE’s leaf-linear intensity is exactly what distinguishes it — the same argument as in the MSE and LogLoss modes, applied to event rates. Note the additive form $\lambda = w^\top \varphi(t)$ differs from the GLM convention $\log \mu = x^\top \beta$ [1]: the intensity is linear, not log-linear, which permits a closed-form Newton step per leaf but requires care to keep $\lambda > 0$.

2 The data contract of `PoissonLegacyBooster`

The Python API (`python/extra_boost_py/poisson_booster.py`) takes five arrays. The semantics below were validated empirically on 2026-07-02 (`test_poisson_semantics.py`); the engine itself documents none of this.

Argument	Shape	Meaning
<code>bjids</code>	$(n,)$ int32	object (cohort) identifier; rows of one object share it
<code>freqs</code>	$(n,)$ f64	event count observed in the row’s time bin; must be > 0
<code>features_inter</code>	(n, p) f64	partition features (constant within an object)
<code>features_extra</code>	(n, d) f64	$\varphi(t_{\text{row}})$ evaluated at the row’s bin center
<code>psi</code>	$(d,)$ f64	$\int_W \varphi(t) dt$ over the <i>whole observation window</i>

So one “row” is *(object, time bin, count)*: an object observed over $W = [0, c]$ partitioned into bins contributes one row per bin. ψ carries the exposure term of Eq. (1): for a leaf model $\lambda = w^\top \varphi$, $\int_W \lambda = w^\top \psi$ per object.

Hard constraints found empirically:

- **zero counts crash the process** (a `mummap_chunk` abort inside the CGO layer) — the benchmark generator therefore guarantees counts ≥ 1 ;
- `predict(F_inter,  $\varphi(t)$ )` returns the intensity **in per-bin count units**, i.e. the expected count of one row, not a rate per unit time;
- only `n_stages=1` produces consistent extra-mode results (Section 4).

3 How training works (`golang/poisson_legacy`)

Training alternates tree building and bias accumulation (`booster.go:Train`): each stage builds one tree against the current prediction (*bias*), then adds the tree’s predictions to the bias. Leaf values of the first tree are used at full scale; later trees are damped by the learning rate (`tree.go:leafFromPrediction`). The interesting part is how leaf weights are computed, and it differs between the first tree and all subsequent ones (`pmatrix.go:WholeLoss` dispatches on `bias == nil`).

3.1 First tree, extra mode: least squares on counts

`wholeLossFirstTreeExtra` solves, for every candidate leaf L ,

$$\left(\sum_{i \in L} \varphi_i \varphi_i^\top + \lambda_{\text{reg}} I \right) w = \sum_{i \in L} y_i \varphi_i, \quad (2)$$

i.e. an *ordinary least-squares regression of the counts on the basis*. This is not the Poisson likelihood — it is its Gaussian surrogate — but it has two convenient properties: it is exact in one shot (no iteration), and if the counts follow a linear trend it reproduces it exactly. This is why the engine’s own Go test (`TestExtraPoissonFirstTree`) sees per-row predictions equal to the training frequencies, and why our validation recovered $\lambda_0(t) = 2 + 2t$ vs $\lambda_1(t) = 4 - 2t$ perfectly with one stage. The fitted w is in **count-per-bin units** because the targets y_i are counts. Note that Eq. (2) does not use ψ at all.

3.2 Split search

For each split candidate the engine evaluates the post-fit loss on both sides using cumulative-sum tables over rows sorted per feature (`Gax`), exactly like the main GBDTE engine; an optional `unbalanced_penalty` discourages extreme splits. Leaf weight vectors for the winning split come out of the same pass (`DeltaUp` / `DeltaDown`).

3.3 Later trees, extra mode: Newton on the Poisson process likelihood

`wholeLossNextTreeExtra` treats the new tree as an *additive correction* $u(t) = w^\top \varphi(t)$ on top of the current intensity $b(t)$ (the bias), and takes one damped Newton step on Eq. (1) around $u = 0$:

$$g = N_L \psi - \sum_{i \in L} \frac{y_i \varphi_i}{b_i}, \quad H = \sum_{i \in L} \frac{y_i \varphi_i \varphi_i^\top}{b_i^2}, \quad w = -H^{-1} g, \quad (3)$$

where N_L counts distinct objects in the leaf (`countObjects`). This is the correct derivative structure for Eq. (1): the exposure term contributes $N_L \psi$ (each object integrates the correction over the whole window), the event term contributes $\sum y \varphi / b$.

4 The inconsistency, precisely

Equations (2) and (3) do not speak the same units.

- The first tree fits $b(t)$ in **per-bin counts**: its stationary condition matches $\sum y_i \varphi_i$, where y_i are counts in bins of width Δ .
- The Newton stage’s stationary condition, $\sum_i y_i \varphi_i / \lambda(t_i) = N_L \psi$, is satisfied by an intensity in **events per unit time**: for a constant true rate r , plugging $\lambda = r$ makes the left side $\approx N_L \int_W \varphi$ only because $\sum_{\text{bins}} (r \Delta) \varphi(t_b) / r = \Delta \sum_b \varphi(t_b) \approx \int_W \varphi$. Plugging the first tree’s count-scale intensity $\lambda = r \Delta$ instead inflates the left side by $1/\Delta$.

So after a perfect first tree, the second stage’s gradient is far from zero — not because the fit is bad, but because the exposure term evaluates it on a different scale. The observed behavior follows: with `n_stages=1` the recovery of a known two-group linear intensity is exact (to sampling noise); with `n_stages=2` every prediction shifts by a systematic offset; with many stages the iteration settles at a fixed point that is *not* the maximizer of the engine’s own likelihood (verified by solving the MLE independently). The strict-xfail test `test_extra_mode_multi_stage_is_consistent` pins this defect.

Fix directions (to be designed before coding).

1. **Make everything per-row Poisson regression on binned counts.** Replace the per-object exposure $N_L\psi$ with a per-row exposure $\varphi(t_i)\Delta_i$ (row i 's bin integrates $\int_{\text{bin}} \varphi \approx \varphi(t_b)\Delta$). Then the likelihood is the standard binned-count Poisson likelihood, units agree with the first tree automatically, ψ and `bjids` become unnecessary, zero counts become legal, and results match what external Poisson boosters optimize. This is the simplest and most conventional repair [2, 5].
2. **Keep the point-process form, fix the first tree.** Fit the first tree by the same Newton objective (starting from a constant initial intensity) so all stages optimize Eq. (1) in rate units, and document that `freqs` rows are event/bin counts while predictions are rates. More faithful to the “true” inhomogeneous process, but more work and still needs a positivity guard for λ .

Either way, a positivity safeguard for leaf intensities matters: the current linear $\lambda(t)$ fitted on $t < 0.6$ can cross zero inside the evaluation window, which is what blew up the Poisson deviance at some depths in the benchmark run.

5 The benchmark generator

`poisson_bench.py` builds the Poisson analog of the paper’s grouped MSE benchmark — “grouped drifting intensity”:

1. **Objects are cohorts.** Each of `n_objects` objects represents a cohort of `exposure` identical units (e.g. a portfolio segment). Cohorts make per-bin counts comfortably positive, which the engine requires.
2. **Latent group $\rightarrow$ intensity.** Object j carries binary partition features encoding a latent group g out of `k_groups` (plus the continuous nuisance feature f_m). The group’s per-unit intensity is linear in time:

$$\lambda_g(t) = \beta_{g,0} + \beta_{g,1} t, \quad \beta_{g,0} \sim U(1, 5), \quad \beta_{g,1} = \text{drift} \cdot u \cdot \beta_{g,0}, \quad u \sim U(-0.8, 0.8),$$

so intensities stay positive on $[0, 1]$ by construction.

3. **Role purity ρ .** As in the MSE family, the actual intensity is a mixture $\rho \lambda_{\text{group}} + (1 - \rho) \lambda_m$, where λ_m depends smoothly on the nuisance m ; at $\rho = 1$ the partition features carry all structure, at $\rho = 0$ none.
4. **Binned observation.** Time $[0, 1]$ splits into `n_bins` equal bins; the row for (object j , bin b) gets count $y \sim \text{Poisson}(\lambda_j(t_b) \Delta \cdot \text{exposure})$, clipped to ≥ 1 (measured distortion at the default exposure: the average zero-probability is 6%; at exposure 60 it vanishes).
5. **Temporal split.** Train on $t < 0.6$, evaluate on $t \geq 0.6$: the model must extrapolate each cohort’s trend into an unseen future window — the same protocol as the MSE ($t < 0.5$) and LogLoss ($t < 0.4$) benchmarks.
6. **Dual form.** The same dataset is exposed (a) as events (`bjids`, `freqs`, `features`, φ , ψ) for GBDTE and (b) as a plain table (rows = object $\times$ bin, target = count, features = partition candidates + t) for LightGBM/XGBoost/CatBoost with their native Poisson objectives. A `lam_true` column stores the true intensity, enabling an oracle upper bound and an intensity-RMSE diagnostic that real data cannot offer.

Evaluation uses the mean Poisson deviance on the future window,

$$D = \frac{2}{n} \sum_i \left[y_i \log \frac{y_i}{\hat{\mu}_i} - (y_i - \hat{\mu}_i) \right],$$

the standard proper scoring rule for count models [2, 7], plus the synthetic-only intensity RMSE against `lam_true`.

6 Where things stand

With the engine restricted to a single stage, GBDTE-Poisson currently *loses* to conventional Poisson boosters on this benchmark (mean deviance 1.02–1.03 vs. 0.85–0.89 for CatBoost/LightGBM; Bayes oracle 0.79–0.80; 10 seeds, Friedman $p = 1.3 \cdot 10^{-20}$), and the wrong-roles ablation is worst (1.45–1.50), as expected. The loss is structural: one tree must simultaneously discover the partition and fit stable leaf trends, with no boosting to correct residual errors — and mis-fitted leaf trends extrapolate badly (occasionally through zero). Fixing the multi-stage update per Section 4 is therefore the prerequisite for any paper-grade Poisson claim; the benchmark, baselines, statistics, and reports are already in place and re-run with one command:

```
PYTHONPATH=python .venv/bin/python scripts/run_article_experiments.py --suite
baselines_poisson --seeds 10
```

References

- [1] P. McCullagh and J. A. Nelder. *Generalized Linear Models*, 2nd ed. Chapman & Hall, 1989.
- [2] A. C. Cameron and P. K. Trivedi. *Regression Analysis of Count Data*, 2nd ed. Cambridge University Press, 2013.
- [3] D. J. Daley and D. Vere-Jones. *An Introduction to the Theory of Point Processes, Vol. I: Elementary Theory and Methods*, 2nd ed. Springer, 2003. (Inhomogeneous Poisson processes and the log-likelihood (1).)
- [4] J. H. Friedman. Greedy function approximation: a gradient boosting machine. *Annals of Statistics* 29(5):1189–1232, 2001.
- [5] E. Ohlsson and B. Johansson. *Non-Life Insurance Pricing with Generalized Linear Models*. Springer, 2010. (Poisson claim-frequency modeling with exposure offsets.)
- [6] M. Denuit, D. Hainaut, and J. Trufin. *Effective Statistical Learning Methods for Actuaries II: Tree-Based Methods and Extensions*. Springer, 2020. (Poisson deviance as the split criterion for count data.)
- [7] M. V. Wüthrich and M. Merz. *Statistical Foundations of Actuarial Learning and its Applications*. Springer Actuarial, 2023. Open access: <https://link.springer.com/book/10.1007/978-3-031-12409-9>.
- [8] Y. Yang, W. Qian, and H. Zou. Insurance premium prediction via gradient tree-boosted Tweedie compound Poisson models. *Journal of Business & Economic Statistics* 36(3):456–470, 2018. arXiv:1508.06378.
- [9] LightGBM documentation: `objective=poisson`. <https://lightgbm.readthedocs.io/en/latest/Parameters.html>.
- [10] XGBoost documentation: `count:poisson`. <https://xgboost.readthedocs.io/en/stable/parameter.html>.
- [11] CatBoost documentation: regression loss functions (Poisson). <https://catboost.ai/docs/en/concepts/loss-functions-regression>.